

■ CodeFlow AI

Simple Working & Technology Guide

Easy English Explanation: What We Used, Why We Used It, and How the Pipeline Works
Clear Technical Manual & Easy Reference

1. What is CodeFlow AI? (Easy Overview)

CodeFlow AI is an automatic AI tool that reviews, tests, and fixes your computer code. When you give it a piece of code, two AI agents work together in a loop:

1. The **Reviewer Agent** finds bugs, security flaws, and mistakes.
 2. The **Developer Agent** automatically writes fixes and runs real unit tests to prove the fix works.
- They repeat this cycle until your code is completely clean and approved.

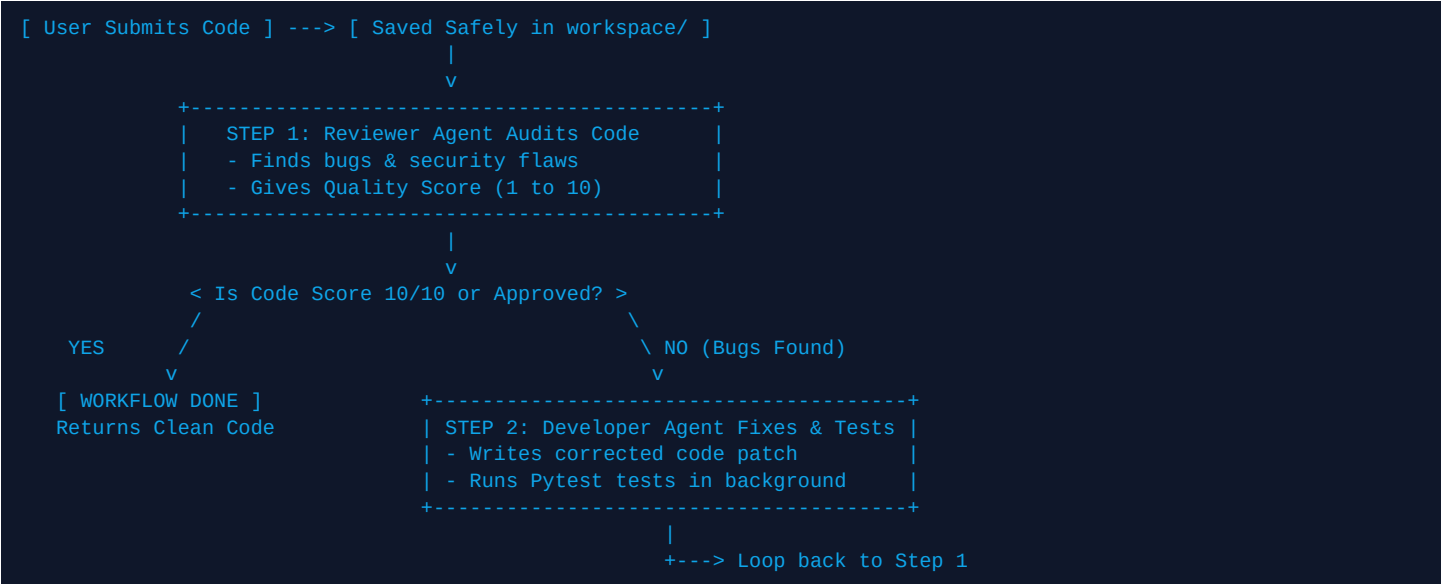
2. Technologies Used: What We Used & Why We Used It

Here is a simple table explaining what technologies were chosen and why each one was selected:

Technology Used	What We Used ('Kiya Use Kiya')	Why We Used It ('Kiu Kiya Use')
LangGraph	LangGraph State Machine (<code>graph/workflow.py</code>).	Why: Normal AI pipelines only move forward in one straight line. But fixing code requires looping back (Review → Fix → Test → Review again). LangGraph allows cyclic loops with clear stop rules.
Groq LLM Engine	Groq high-speed AI API running LLaMA 3.3 70B.	Why: Groq runs AI extremely fast (300+ words per second). This allows multi-step review and patching cycles to finish in a few seconds instead of minutes.
FastAPI	Python web server (<code>api.py</code>).	Why: FastAPI is very fast, handles multiple tasks at the same time, and supports real-time event streaming to the web screen.
Server-Sent Events (SSE)	Streaming response format (<code>text/event-stream</code>).	Why: SSE streams live updates to the screen step-by-step while the AI is thinking. It is simpler and lighter than complex WebSockets.
Pydantic	Structured data schemas (<code>schemas/</code>).	Why: AI outputs can sometimes be messy. Pydantic forces the AI to output clean, structured data with scores, line numbers, and bug descriptions.
Pytest Runner	Isolated testing tool (<code>tools/test_tools.py</code>).	Why: Never trust AI text alone! Pytest actually executes real unit tests on the fixed code to verify that the bugs are truly solved.
Safe Folder Sandbox	Path security guard (<code>get_safe_path</code>).	Why: Keeps all code files strictly inside the <code>workspace/</code> folder, preventing any harm to main system files.
React + Vite	Web Dashboard Interface (<code>frontend/</code>).	Why: Gives users a modern, clean web screen with live score tracking and side-by-side code diff views.

3. Step-by-Step Pipeline: How It Works from Start to Finish

Here is the easy step-by-step process of how your code moves through the system:



Step 1: Receiving & Saving the Code

- You send your code string or file through the web screen.
- The system safely writes your file into the `workspace/` folder using path validation.

Step 2: Reviewer Agent Checks the Code (`reviewer\_node`)

- The Reviewer AI reads your code and looks for bugs, security holes (like SQL injection or bad file paths), and performance problems.
- It rates your code quality from **1 to 10** and lists all issues with line numbers.
- **Auto-Repair Protection:** If the AI response is cut off, our system automatically adds missing brackets (`` and ``) so the application never breaks.

Step 3: Checking the Result (`should\_continue`)

- If the code is rated **APPROVED** (10/10 score with no major bugs), the system stops and gives you the final code.
- If the code has bugs, it moves to the Developer Agent.
- If it has already tried 3 times (maximum loop limit), it stops safely to avoid infinite loops.

Step 4: Developer Agent Fixes & Runs Tests (`developer\_node`)

- The Developer AI reads the bug list from the Reviewer AI.
- It rewrites the code to fix all reported bugs.
- It saves the fixed code and executes real **Pytest** unit tests in the background to count how many tests pass or fail.
- It sends the fixed code back to the Reviewer AI to check again.

4. Why Use Two Separate AI Agents? (Reviewer vs Developer)

Instead of using one single AI prompt, we split the work between two distinct agents:

AI Agent	Main Job / Responsibility	Why Separation is Important
Reviewer Agent (The Strict Inspector)	Audits code objectively. Finds security flaws and rates code from 1 to 10.	Prevents Self-Bias: If the same AI writes and checks code, it will always say its code is perfect! Using a separate inspector agent ensures strict, unbiased reviews.
Developer Agent (The Fixer & Tester)	Reads bug reports, applies targeted code patches, and runs automated Pytest tests.	Focuses on Real Execution: Works like a real programmer by saving code files and running tests to prove the fix works.

5. Real-Time Stream Events (How Data Reaches Your Screen)

While the AI is working, FastAPI sends real-time events directly to your web dashboard screen:

Event Name	When Is It Sent?	What Shows on Your Screen?
`type: reviewer`	Sent right after the Reviewer AI finishes auditing code.	Displays code score badge (e.g. 5/10), decision status, and detailed defect table with line numbers.
`type: developer`	Sent right after the Developer AI fixes code and runs unit tests.	Displays side-by-side code diff comparison (original vs fixed) and Pytest pass/fail metrics.
`type: done`	Sent when code is fully approved or maximum loop limit is reached.	Shows green success banner, stops loading animations, and enables final code download.
`type: error`	Sent if an API key is missing or system error happens.	Displays clear error message popup to explain what went wrong.

6. Quick Summary Table

Component	Technology Used	What It Does (Simple)
Orchestration	LangGraph	Controls the loop between Reviewer and Developer AI.
AI Intelligence	Groq LLaMA 3.3 70B	Provides super-fast AI model responses.
Web Server	FastAPI	Handles web API endpoints and live data streaming.
Testing Engine	Pytest	Runs actual unit tests to prove code fixes work.
File Security	Path Sandbox	Protects system files by keeping edits inside workspace.
User Dashboard	React + Vite	Provides a beautiful visual UI with live code diffs.

Summary Conclusion:
CodeFlow AI is an automated, self-healing code review system. It uses fast Groq AI, LangGraph looping, and Pytest unit tests to automatically fix bugs, enforce security, and deliver clean, verified code.